



# Application Penetration Testing Report

Base URL: [traceability.apeda.gov.in/meatnet](https://traceability.apeda.gov.in/meatnet)

Report Date: 16-04-26 04:25 pm

Application Security and Audit Division  
(Application Security Group)  
National Informatics Centre  
A – Block, CGO Complex, Lodhi Road, New Delhi - 110003.

## Contributions

| S. No. | Name                    | Role     |
|--------|-------------------------|----------|
| 1.     | Harmanjit Singh Dhillon | Auditor  |
| 2.     | Sudhir Kumar Verma      | Reviewer |

## Key Findings

### Vulnerability List

| Assign Number: A20260121020                 |                                                                                                                             |                                                                      |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Base URL: traceability.apeda.gov.in/meatnet |                                                                                                                             |                                                                      |
| S. No.                                      | Incident URL                                                                                                                | Vulnerability Name                                                   |
| 1.                                          | <a href="https://traceability.apeda.gov.in/meatnet">https://traceability.apeda.gov.in/meatnet</a>                           | <a href="#">Login Request Replay Attack</a>                          |
| 2.                                          | <a href="https://traceability.apeda.gov.in/Meatnet/Js/jquery-">https://traceability.apeda.gov.in/Meatnet/Js/jquery-</a>     | <a href="#">Framework Disclosure</a>                                 |
| 3.                                          | <a href="https://traceability.apeda.gov.in/meatnet">https://traceability.apeda.gov.in/meatnet</a>                           | <a href="#">Improper Implementation of Security Response Headers</a> |
| 4.                                          | <a href="https://traceability.apeda.gov.in/Meatnet/Js/NewMd5.js">https://traceability.apeda.gov.in/Meatnet/Js/NewMd5.js</a> | <a href="#">Weak Hashing Algorithm</a>                               |

## 1. Vulnerability Name: Login Request Replay Attack

Severity:

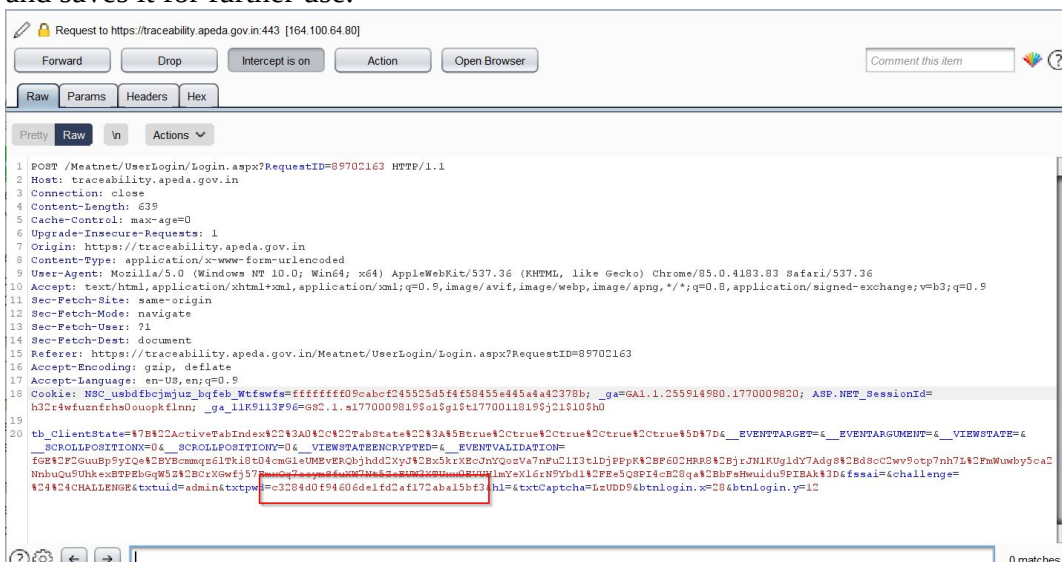
**Medium**

Proof of Concept:

Step #1: An attacker navigates to the application, enters the wrong credentials and clicks on Login button as shown below:



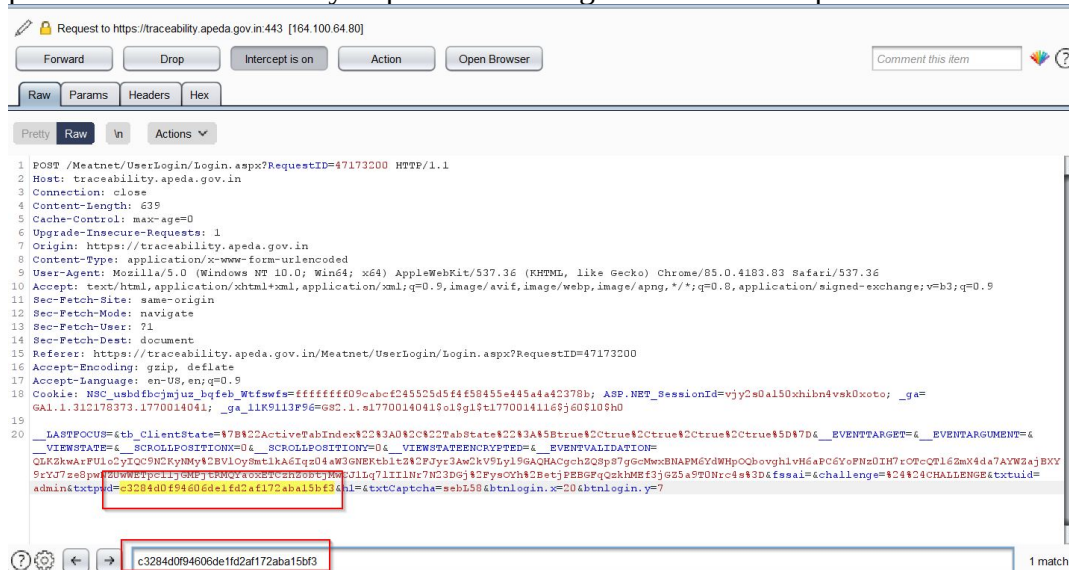
Step #2: The attacker intercepts the request using an HTTP Interceptor, copies wrong password hash and saves it for further use.



Step #3: An attacker again navigates to the application, enters the wrong credentials and clicks on Login button as shown below:



**Step #4:** The attacker intercepts the request using an HTTP Interceptor, and notices that hashed password is same in every request confirming that no salt is implemented.



### Description:

The successful login request can be replayed, allowing unauthorized access without re-authentication

### Impact:

An attacker can intercept and reuse a valid login request from an authenticated user, bypassing authentication controls to gain unauthorized access to the application as the victim.

### Recommendations:

It is recommended to implement the validation against the session ID or token generated for the login page which is invalidated after use. It is recommended to use salted hashing (SHA-2 or higher) or a robust encryption algorithm to securely transform the password into randomized ciphertext. The salt or encryption keys used for hashing or encryption should be randomly generated on the server side. The salt or keys should be regenerated for each successful or unsuccessful attempt.

### Other Links:

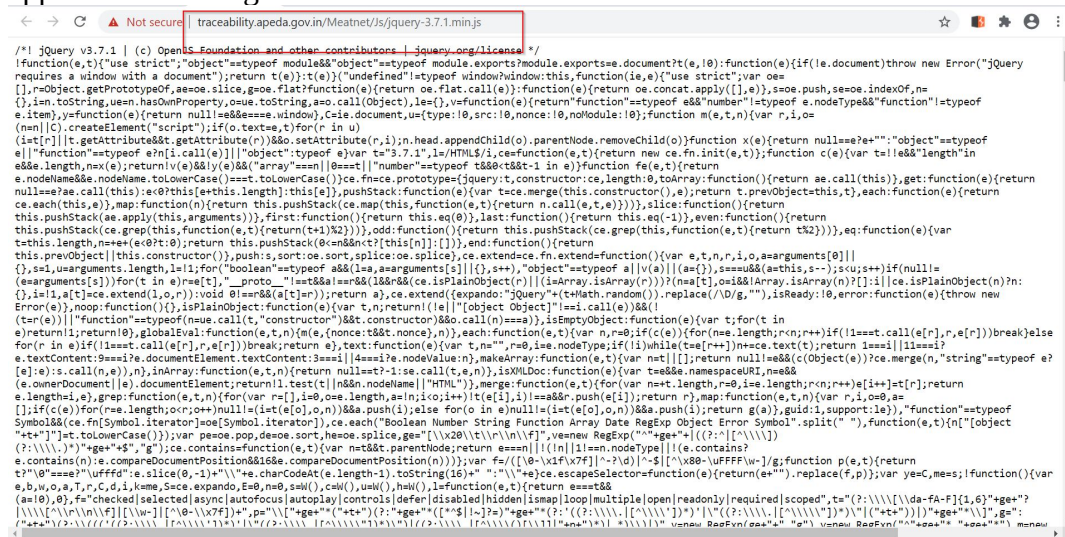
## 2. Vulnerability Name: Framework Disclosure

### Severity:

**Low**

### Proof of Concept:

Step #1: From the below screenshot, it can be observed that the framework name and version of the application is being disclosed.



### Description:

The application discloses the framework details like name and version to the end user.

### Impact:

The application discloses the framework details such as name and version to the end user. A malicious user can use this information to launch further targeted attacks.

### Recommendations:

It is recommended that the application should not display the framework details to the end users.

### Other Links:

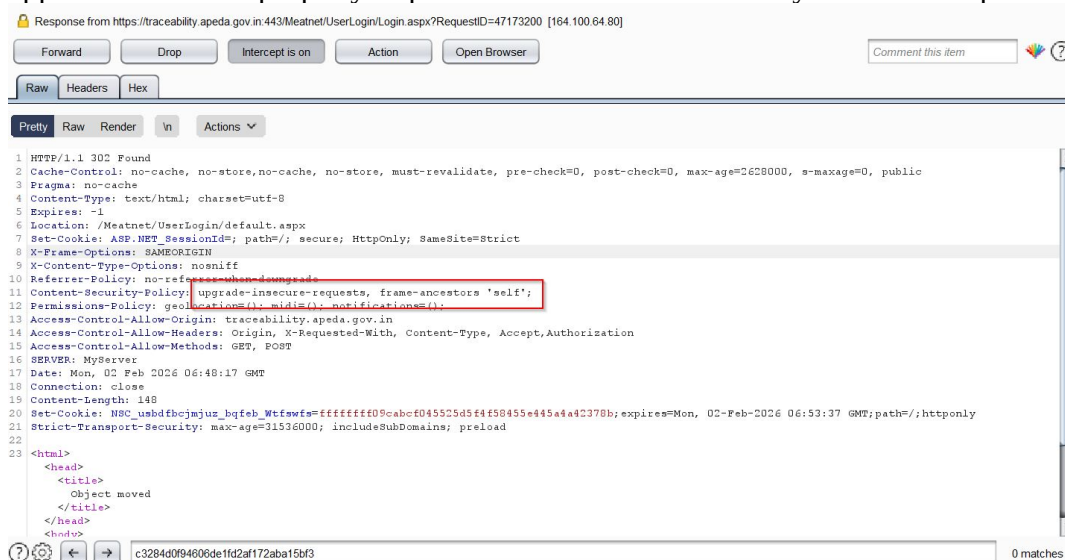
### 3. Vulnerability Name: Improper Implementation of Security Response Headers

#### Severity:

**Low**

#### Proof of Concept:

Step #1: An attacker navigates to the application, captures the response and observed that the application has not properly implemented some of the security header in response as shown below:



#### Description:

The application has improperly implemented security response headers

#### Impact:

The application has improperly implemented security headers, as an attacker may take advantage of this vulnerability and may perform attacks like clickjacking and Cross Site Scripting.

#### Recommendations:

The application should implement Content-Security-Policy: script-src 'self'; in the response header. It is recommended that the application should properly implement 'Strict-Transport-Security: max-age; includeSubDomains;' in the response header. The application should implement 'X-XSS-Protection: 1; mode=block' in the response header. The application should properly implement 'X-Frame: Deny' in the response header.

#### Other Links:

## 4. Vulnerability Name: Weak Hashing Algorithm

### Severity:

**Low**

### Proof of Concept:

INSTANCE#1: Step #1: From the below screenshot, it can be observed that the application using weak hashing algorithm.

```

/*
 * md5.js v1.0b 27/06/96
 *
 * Javascript implementation of the RSA Data Security, Inc. MD5
 * Message-Digest Algorithm.
 *
 * Copyright (c) 1996 Henri Torgemane. All Rights Reserved.
 *
 * Permission to use, copy, modify, and distribute this software
 * and its documentation for any purposes and without
 * fee is hereby granted provided that this copyright notice
 * appears in all copies.
 *
 * Of course, this soft is provided "as is" without express or implied
 * warranty of any kind.
 *
 * $Id: md5.js,v 1.1.1.1 2000/04/17 16:40:07 kk Exp $
 */

function array(n) {
  for (i = 0; i < n; i++) this[i] = 0;
  this.length = n;
}
/* Some basic logical functions had to be rewritten because of a bug
in
 * Javascript.. Just try to compute 0xffffffff >> 4 with it..
 * Of course, these functions are slower than the original would be,
but
 * at least, they work!
 */

function integer(n) { return n % (0xffffffff + 1); }

function shr(a, b) {
  a = integer(a);
  h = integer(h);
}

```

INSTANCE#2: Step #1: From the below screenshot, it can be observed that the application using weak hashing algorithm.

```

/*
 * A JavaScript implementation of the RSA Data Security, Inc. MD5 Message
 * Digest Algorithm, as defined in RFC 1321.
 * Version 2.1 Copyright (C) Paul Johnston 1999 - 2002.
 * Other contributors: Greg Holt, Andrew Kepert, Ydnar, Lostinet
 * Distributed under the BSD license
 * See http://pajhome.org.uk/crypt/md5 for more info.
 */

/*
 * Configurable variables. You may need to tweak these to be compatible with
 * the server-side, but the defaults work in most cases.
 */
var hexcase = 0; /* hex output format. 0 - lowercase; 1 - uppercase */
var b64pad = ""; /* base-64 pad character. "=" for strict RFC compliance */
var chrsz = 8; /* bits per input character. 8 - ASCII; 16 - Unicode */

/*
 * These are the functions you'll usually want to call
 * They take string arguments and return either hex or base-64 encoded strings
 */
function hex_md5(s) { return bin2hex(core_md5(str2binl(s), s.length * chrsz)); }
function b64_md5(s) { return bin2b64(core_md5(str2binl(s), s.length * chrsz)); }
function str_md5(s) { return bin2str(core_md5(str2binl(s), s.length * chrsz)); }
function hex_hmac_md5(key, data) { return bin2hex(core_hmac_md5(key, data)); }
function b64_hmac_md5(key, data) { return bin2b64(core_hmac_md5(key, data)); }
function str_hmac_md5(key, data) { return bin2str(core_hmac_md5(key, data)); }

/*
 * Perform a simple self-test to see if the VM is working
 */
function md5_vm_test()
{
  return hex_md5("abc") == "900150983cd24fb0d6963f7d28e17f72";
}

/*
 * Calculate the MD5 of an array of little-endian words, and a bit length
 */

```

### Description:

The application uses a weak hashing algorithm to hash the sensitive data

### Impact:

The application uses a weak hashing algorithm to hash the sensitive data. As a result, an attacker may be able to extract passwords and perform further targeted attacks.

### Recommendations:

Application must use strong hashing algorithm like SHA256/512. The information must be hashed with random salt and salt should be unique for every request. Salt should be generated at server side and properly validated.

[Other Links:](#)

<https://traceability.apeda.gov.in/Meatnet/Js/encmd5.js>